

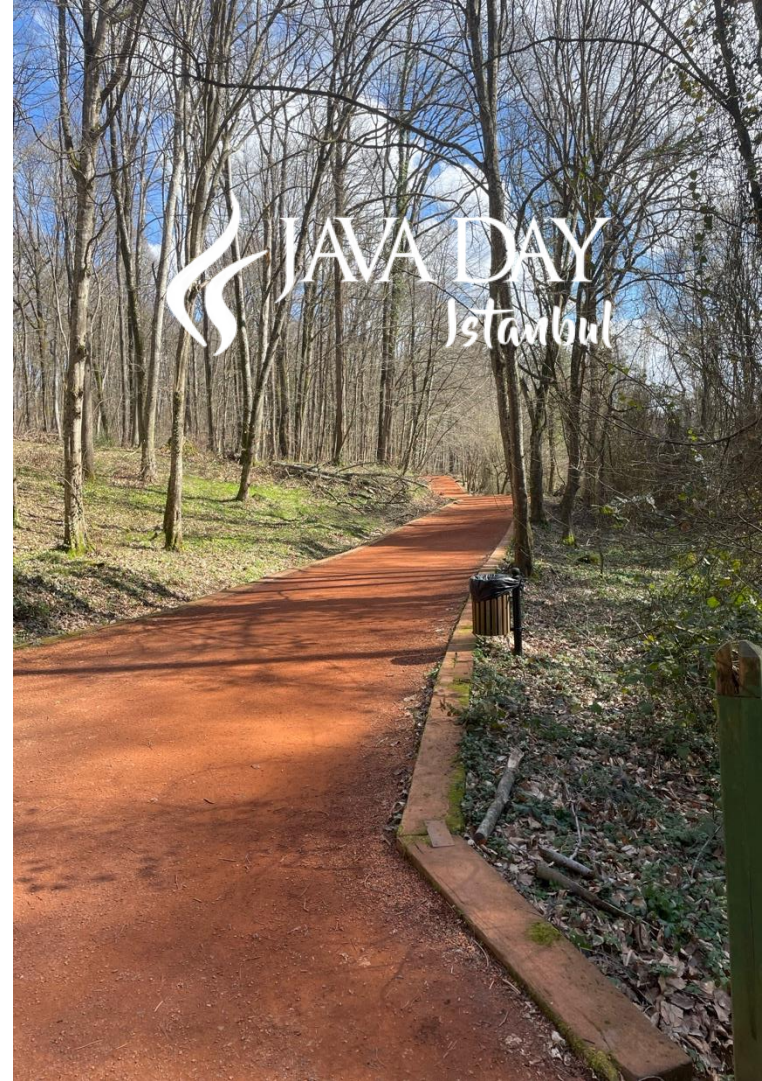
Java Day Istanbul 2026

LLM-Powered Unit Testing

Writing Reliable Java Tests with AI Assistance

İbrahim ATAY, Senior Architect

in [linkedin.com/in/ibrahimatay](https://www.linkedin.com/in/ibrahimatay) X x.com/ibrahim_atay ● github.com/ibrahimatay



Gündem

01

Problem Tanımı

Kurumsal unit testing eksikliği

02

Çözüm Yaklaşımı & Akış

LLM tabanlı test üretim metodu ve diyagram

03

Teknik Zorluklar

Karşılaşılan engeller ve çözümler

04

Prompt Yapısı & Örnek

Kullanılan prompt tasarımı ve örnek çıktı

05

Sayısal Sonuçlar

3 uygulamada elde edilen kazanımlar

06

Çıkarımlar

Temel takeaway'lar



Java + AI
Test Üretimi

JAVA DAY
İstanbul

Problem Tanımı

Mevcut Durum

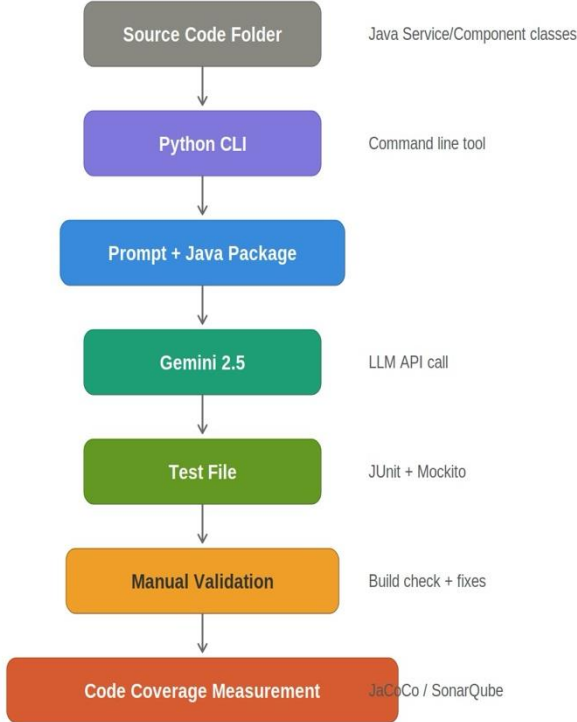
- Unit Testing kurumsal bir kalite kültürü olarak yaygınlaştırılmalı
- Legacy uygulamaların test kapsamına alınması gerekli
- Çok sayıda ekranı olan, aktif kullanılan uygulamalarda sonradan test yazımı zorlu
- Zaman, koordinasyon ve teknik uygunluk açısından yüksek efor



Çözüm Yönelimi

- LLM tabanlı araçlardan yararlanma
- Ortak mimariye uyumlu Unit Testing LLM altyapısı
- İş katmanı sınıfları prompt ile girdi olarak veriliyor
- JUnit + Mockito ile test taslakları üretimi

Çözüm Yaklaşımı — İş Akışı



Temel Bileşenler



Python CLI

Kaynak kod paketini parametre olarak alır



Prompt Şablonu

Yapılandırılmış prompt ile Gemini 2.5'e istek gönderir



Test Dosyaları

JUnit 5 + Mockito test taslakları üretilir



Doğrulama

Manuel review + build check + fix döngüsü



Coverage İzleme

JaCoCo + SonarQube ile kalite ölçümü

Teknik Zorluklar & Çözümler

Test Edilebilirlik

Kod tabanı unit test'e uygun tasarlanmamıştı → Ek soyutlama katmanları geliştirildi

Static Mock Sorunu

Framework'teki static nesneler mock edilemiyordu → Özel teknik yaklaşımlar uygulandı

İş Kuralı Bağlamı

Teknik yapı yeterli olmadı → Manuel gözden geçirme ve iyileştirme gerekti

Token Kapasitesi

Tüm proje bağlamı aktarılamıyor → Sınıf/metod seviyesinde kontrollü test üretimine odaklanıldı

Prompt Tasarımı — Temel Prensipler

Rol Tanımı

Expert Java/Spring Boot test yazarı rolü

Girdiler

BaseTest sınıfı + hedef service/component dosyası

Koşullu Mantık

Native SQL tespit → skip, değilse → test üret

Kurallar

JUnit 5 only, no AssertJ/Hamcrest

Kısıtlamalar

No Spring context, no reflection, no narrative text

Çıktı Formatı

Derlenebilir Java kodu — markdown yok, placeholder yok

Prompt Anatomisi

JAVA_SPRING_TEST_FILE_PROMPT_TEMPLATE

1

BAĞLAM

Role Expert Java/Spring Boot test yazarı — JUnit 5 + Mockito

Inputs base_test_class_content + file_content

Skip @Query(nativeQuery), JdbcTemplate → skip

2

YAPISAL KURALLAR

Base Class @ExtendWith, BaseTest'ten türet, mock'ları paylaş

SUT private @InjectMocks — manuel instance yok

Setup @BeforeEach ile ortak stub'lar

3

MOCK KATMANI

DI & Mock @Mock tüm bağımlılıklar, when().thenReturn() stub

Repository findById, findAll, save — Optional.of & empty

Test Data Builder pattern, deterministic, List.of()/Set.of()

4

DOĞRULAMA

Assertions JUnit 5 only: assertEquals, assertNotNull, assertThrows

Verify verify(mock, times(n)), ArgumentCaptor

State State transitions, @Version, audit fields

5

TEST KAPSAMI

Coverage Tüm public metotlar: happy path + edge case + exception

Pattern AAA: // Arrange → // Act → // Assert

Naming shouldDoSomething_WhenCondition

6

ÇIKTI KONTROLÜ

Yasak AssertJ, Hamcrest, @SpringBootTest, reflection, narrative

Logger Mock'lama — Logger side effect, test edilmez

Output Derlenebilir Java — markdown yok, placeholder yok

Sayısal Sonuçlar

Uygulama	Başlangıç	Sonuç	Verimlilik	Test Metot	Manuel (sa)	AI (sa)
App-1	0%	27.7%	10.6x	224/503	70	7
App-2	3.2%	25.2%	8.2x	320	106	13
App-3	5.2%	32.1%	2.2x	96	32	14

10.6x

Maks. Verimlilik

640

Toplam Test Metodu

~174 sa

Tasarruf Edilen Efor

Öne Çıkan Kazanımlar

10.6x

Verimlilik Artışı

App-1'de manuel test eforu ile karşılaştırıldığında elde edilen maksimum kazanım

640 test metodu · **174 sa** tasarruf

Legacy projelerde test kapsamı genişledi

0% → %27.7 (App-1), %3.2 → %25.2 (App-2), %5.2 → %32.1 (App-3) — daha önce hiç test yazılmamış kod tabanlarında somut ilerleme.

Mühendis başlangıç eşiği düştü

LLM, boilerplate test yapısını otomatik üreterek geliştiricilerin iş kuralı ve edge case'lere odaklanmasını sağladı.

Tutarlı test standartları sağlandı

Yapılandırılmış prompt ile AAA pattern, naming convention (@Test shouldX_WhenY) ve JUnit 5 assertion kullanımı garanti altına alındı.

Kurumsal ölçekte tekrarlanabilirlik kanıtlandı

Aynı CLI aracı ve prompt şablonu 3 farklı uygulamada çalıştırılarak yaklaşımın ölçeklenebilirliği doğrulandı.

AI ile Çalışma İpuçları

Doğal Dilinizi Kullanın

Kendinizi en iyi ifade ettiğiniz dilde AI ile iletişim kurun.

Her Çıktıyı İnceleyin

AI çıktısını bir kod incelemesi yapar gibi dikkatle review edin.

Bağlam Sağlayın

Projede daha önce yapılmış içerikleri AI'a açıklayın.

Dosyaları Çalışma Alanına Ekleyin

AI'ın dosyaları düzenlediğini fark edebilirsiniz.

Sürüm Kontrolü Şart

Görev bazlı ilerleyin, commit yapın — hata durumunda geri dönüş sağlayın.

Temel Çıkarımlar

1

LLM Hızlandırır — İnsan Denetimi Şart

AI testleri başlangıç noktasıdır. Edge case ve iş kuralları için manuel review gerekli.

2

Legacy Kod İçin Büyük Fırsat

Sıfırdan test yazmanın en zor olduğu projelerde 2x–10x verimlilik artışı.

3

Prompt Tasarımı Kritik

Kural bazlı, yapılandırılmış şablonlar tutarlı ve derlenebilir çıktı sağlar.

4

İteratif Yaklaşım Benimseyin

JaCoCo + SonarQube ile her iterasyonda kaliteyi artırın.

5

AI, Geliştiriciyi Değerli Kılar

Boilerplate'den kurtularak iş kuralı ve tasarıma odaklanın.

Teşekkürler

Sorular & Tartışma

*“Ne kadar bilersen bil, söylediklerin
karşındakinin anlayabildiği kadardır.”*

— **Mevlana**

in [linkedin.com/in/ibrahimatay](https://www.linkedin.com/in/ibrahimatay) X x.com/ibrahim_atay ● github.com/ibrahimatay

Java Day Istanbul 2026

